

IOSG Crypto Tech Dashboard — Technical Handover

IOSG Crypto Tech Dashboard — Technical Handover

0. Executive summary
1. Product overview
2. Three-phase plan synthesis
3. Functional modules — feature inventory
4. Data research workflow & data sources
5. Scoring methodology & experiments
6. Complete API surface
7. Local-mode quickstart (no Docker)
8. Docker deployment (production)
9. Operations & monitoring
10. Known limits & open items
11. Repo layout reference
12. Glossary
13. Contacts & escalation

IOSG Crypto Tech Dashboard — Technical Handover

Status: Production preview (v2.5.0 / commit baseline b96f7e3 + Phase-3 polish) **Author of this hand-off:** Zequn (IOSG quant research) **Successor / new maintainer:** incoming colleague — full ownership transfer **Date:** 2026-05-26 **Repo root for code:** `crypto-tech-dashboard-2nd-try-v2.0/crypto-tech-dashboard/` **Companion docs** (in the same parent folder): - `PLAN_Phase1_EN.md` — Phase-1 implementation plan (English translation) - `PLAN_Phase2_EN.md` — Phase-2 expansion plan (English translation) - `PLAN_Phase3_EN.md` — Phase-3 polish & launch plan (English translation) - The original Chinese plans are kept in the parent directory as `*_zh.md` backups.

0. Executive summary

The **IOSG Crypto Tech Dashboard** is an internal screener that lets IOSG and portfolio analysts triage roughly 240 names (200 crypto by CoinGecko market cap + ~40 US crypto-related stocks) by a battery of 12 technical-indicator families plus a

Tier-A linear-weighted **Overall** composite score. It is a single-machine FastAPI + vanilla-JS app, packaged as a Docker container that runs on a Mac mini and is exposed publicly through a Cloudflare Named Tunnel.

The product has been through three planning rounds:

1. **Phase 1** — original ship: data layer, 12 indicators, Trend / Reversal scores, cross-sectional + 2y/3y time-series percentiles, TradingView-style dark theme, daily APScheduler refresh (originally 08:30 Asia/Shanghai; later moved to 09:00 in Phase 3).
2. **Phase 2** — 11 user-driven improvements: composite Overall score (Tier A theory weights + Tier B Ridge regression), light mode, full English localization, market-cap / liquidity / 30-day volume panel, indicator robustness backtests, US stocks via yfinance, history backfill to 2020-01-01, green-folder portability, container-safe DATA_DIR, per-token data-coverage metadata.
3. **Phase 3** — go-to-LP polish: access-code gate, legal footer, boot-time + 09:00 cron auto-refresh with self-heal, US-stocks OHLC endpoint fix, `asset_class` naming unification, quant-disclaimer copy rewrite (no “AI/ML” wording), 5 visual fixes from the Milan-designer audit, PM patch items (favicon, 404 graceful, case-insensitive URL token).

This document is everything the new maintainer needs to keep it running and to make safe edits.

1. Product overview

1.1 Job-to-be-done

Field	Value
Primary user	IOSG investor / quant researcher / portfolio-company CTO
Decision supported	“Of today’s top-200 crypto and 40 crypto-equity universe, which 10-20 names are technically setting up for trend continuation or mean-reversion bounce , and how does today’s reading rank in the token’s own 2y / 3y history?”
Success state	Open the page, see top-of-list candidates within 3 seconds, click into a token, glance at 12 indicator panels + Trend / Reversal / Overall breakdown, decide whether to add it to a deeper-research queue.
Explicit non-goal	This is a screenner , not a trading terminal. No order routing, no portfolio P&L, no real-time tick stream.

1.2 Daily flow (operator's mental model)

09:00 Asia/Shanghai APScheduler cron fires `run\_daily\_update`
(crypto, CCXT waterfall)
09:05 Asia/Shanghai APScheduler cron fires
`run\_stocks\_daily\_update` (yfinance)
every hour at :30 Self-heal cron: if last update $\geq$ 1 day
stale, retry (4h cooldown)
boot Lifespan re-checks freshness; if stale,
fires background refresh
on demand operator hits POST /api/system/refresh
(also alias /api/refresh)

OHLCV files live on the host at `local_data/ohlc/<cg_id>.csv` (one CSV per token), atomically appended on each daily tick. The container mounts the host `local_data/` so refreshes persist.

2. Three-phase plan synthesis

The three Chinese-language plan documents are translated in full as `PLAN_Phase{1,2,3}_EN.md`. The condensed roadmap is below; treat it as a guide to *what was decided and why*, not a substitute for the detailed plans.

2.1 Phase 1 — original implementation plan

Goal: turn an existing Jupyter notebook
(`1_First_or_not_run___Tech_Fac_Price_Mcap_volumeData.ipynb`) into a single-Mac deployable web app for Top-200 crypto technical analysis.

Decisions that still constrain the codebase:

- **Tech stack frozen at:** FastAPI + pandas + CCXT (backend), vanilla JS + TradingView Lightweight Charts v4 (frontend), local CSV cache (no DB), APScheduler cron, python-dotenv. No Node.js build step.
- **Data source policy:**
 - OHLCV → CCXT waterfall across exchanges (originally 4: Binance → OKX → Bybit → Gate.io; Phase-2 extended to 8 with Coinbase / Kraken / KuCoin / Bitstamp).
 - Token universe + market cap → CoinGecko Pro /coins/markets (3 pages $\times$ 250 = 750 candidates, filtered to Top-200 after exclusion list).
 - Close-only fallback → CoinGecko /market_chart/range for tokens with no exchange listing.
 - Volume + High/Low always from the **same** exchange call (no cross-source mixing).
- **12 indicator families** ported from notebook `compute_features` (final cell). The formulas are canonical and **must not drift**; see §5 below.
- **Scoring:** 9 trend signals + 7 reversal signals, equal-weighted cross-sectional percentile blend, output 0-100.
- **Time-series percentile:** 2y and 3y rolling-window self-history percentile per token (the marquee feature; requires $\geq$ 730 days of OHLCV per token).

- **Frontend:** TradingView dark theme, candle chart synchronized with 12 indicator panels via `subscribeVisibleLogicalRangeChange` master-slave.

Status at end of Phase 1: shippable but flagged C+ in the Google-PM audit (PRODUCT_REVIEW.md). Critical issues fixed before Phase 2 closed: OHLCV pagination (was capped at ~300 days; rewritten to walk the since cursor for full 3y history), searchable token combobox, sidebar rank rows enriched with symbol+name, indicator-panel legends, score tooltips.

2.2 Phase 2 — eleven user-driven improvements

User submitted 11 prompts after Phase 1. The team then resolved 16 clarification questions and structured the work into **Phase 2A → 2B → 2C → 2D** sprints. Highlights, in user-priority order:

#	User ask	What was built
1	Score UX is confusing — explain Trend / Reversal, show rank	<code>/api/scoring/explainer</code> (title, one-line summary, formula, signal table, strengths, weaknesses, interpretation bands); <code>rank_in_universe_trend/reversal/overall: int (1..N)</code> added to <code>/api/scores/{id}</code> .
2	Build a composite score	<code>backend/scoring/overall_score.py</code> . Tier-A weights are finance-theory priors: $0.40 \text{ Trend} + 0.25 \text{ Reversal} + 0.15 \text{ Breadth} + 0.10 \text{ Risk} + 0.05 \text{ TS-Trend-2y} + 0.05 \text{ TS-Reversal-2y}$. Tier-B is <code>scripts/train_tier_b.py</code> (Ridge regression, walk-forward, accept gate $\rho \geq \text{Tier-A} + 0.02$).
3	Light mode	21 light CSS vars; two-layer off-white (<code>#F0F3FA</code> canvas + <code>#FFFFFF</code> cards), accent colours darkened to clear WCAG AA. Theme toggle button persists via <code>localStorage</code> ; charts re-tint without rebuild (state preservation).
4	All-English UI	All Chinese strings purged from <code>frontend/</code> and <code>backend/</code> (<code>.py</code> / <code>.js</code> / <code>.html</code> / <code>.css</code>). Docs kept Chinese.
5	Market overview panel	<code>/api/market_overview/{cg_id}</code> returns mcap rank, total mcap, 24h volume (also 30d avg volume), liquidity proxy. Rendered in a card next to the score gauges.
6	Indicator reliability backtests	<code>backend/backtest/universe_robustness.py</code> runs 9 canonical strategies (RSI 30-50, MACD signal cross, KDJ oversold cross, Bollinger lower-band, SMA/EMA golden cross, momentum breakout, z-score reversion, price appreciation + volume) on every token. Aggregates median Sharpe, % positive, worst/best case. Assigns a reliability badge (<code>reliable</code> / <code>caveats</code> /

#	User ask	What was built
		unreliable). Cache-keyed by sha256(ohlcv files).
7	US stocks	backend/data/yfinance_client.py mirrors the CoinGecko client API. Universe = 40 US tickers in local_data/metadata/stocks_universe.csv (~38 typically return usable yfinance history; 2 may be delisted or symbol-renamed at any time, so /api/scores?asset_class=us-stock reports ~38 ranked names). Cross-section ranking is partitioned by asset_class so crypto and stocks are ranked separately. Default landing token = CRCL.
8	Tooltips everywhere	12 panel header tooltips + 16 score-component tooltips + 7 parameter labels. Native title= for headers; 80-line lightweight popover for the score-breakdown rows (200ms hover delay, persistent inside the popover, optional “Methodology →” link).
9	Overall composite headline panel	Full-width hero card on top with a 240 px gauge + 56 px score; below it the existing Trend + Reversal cards (170 px gauges, 38 px numbers). Hero has a 2 px blue left-accent border and a COMPOSITE 9 px badge.
10	Local data + incremental write + corruption recovery	Fixed the hard-coded LocalStore(PROJECT_ROOT/"local_data") bug → now honours DATA_DIR. Added backend/data/integrity.py boot-time verification, local_data/quarantine/ for unreadable CSVs, Fetcher.repair_token() + POST /api/admin/repair/{cg_id} (localhost-only). New script scripts/pack_green_folder.sh for portable bundling.
11	History back to 2020-01-01	.env HISTORY_DAYS=2326. scripts/run_history_extension.py walks each token’s existing earliest date back via CCXT since= pagination; falls back to CoinGecko close-only on Tier-4. Per-token local_data/metadata/data_coverage.json records earliest_date, listing_date, real_ohlcv_from, close_only_windows, and a tier breakdown. Surfaced in the UI as a collapsible “Data Coverage” pill in the score area.

Quant policy decisions baked in:

- Stocks and crypto are **separate cross-sections**. Mixing them would conflate vol regimes.

- The Tier-B Ridge model is wired but its **accept gate** can fail. When it fails, /api/scoring/tier_b still returns the holdout payload (accept: false) and the UI silently falls back to Tier-A — the Tier-B toggle is hidden by frontend/js/app.js renderTierBBanner().
- A third “calibrated” mode (backend/scoring/calibrated_weights.py) sizes weights by |Fama–MacBeth ρ | per sleeve and respects sign. Same fallback: if the calibration file is missing/invalid, the system returns Tier-A.

2.3 Phase 3 — eight modules to ship a credible preview to LPs

After the Phase-2 four-way audit (quant engineer / senior analyst / Milan designer / Google PM), the team scoped a 4-5 day finishing sprint:

Module	Outcome
1. Access gate	frontend/login.html + frontend/js/auth.js. Password = IOSG (uppercase 4 letters), hard-coded, localStorage memoised. Front-end-only — no backend token check (acceptable for a controlled preview).
2. Legal footer	Sticky bottom footer: copyright, version, “Last data refresh: X ago”, multi-line disclaimer (“research preview, not investment advice, prices may be delayed”). Reads /api/system/health for the version.
3. Auto-refresh + 09:00 cron	Lifespan boot-time freshness check + every-hour :30 self-heal with 4-hour cooldown + APScheduler cron at UPDATE_HOUR=9 / UPDATE_MINUTE=0 (crypto) and 09:05 (stocks). misfire_grace_time=6h + coalesce=True + max_instances=1 so a Mac mini that slept through 09:00 catches up exactly once on wake. Topbar shows “Updated X hours ago” (relative time, full ISO in title hover). <i>Note:</i> the hourly self-heal and APScheduler robustness settings were added during Phase-3 execution beyond the original Phase-3 plan text; both are verified in backend/main.py.
4. US-stocks OHLC fix	Validator + data_service.get_ohlc() made case-tolerant so /api/ohlc/MSTR returns the CSV at local_data/ohlc/MSTR.csv correctly. Verified on MSTR / COIN / CRCL / MARA.
5. asset_class naming	Unified to us-stock (hyphen) across backend + frontend. ? asset_class=stock returns empty (no silent fallback).

Module	Outcome
6. Quant disclaimer copy	Banner changed from “Theory weights · research-only” → “Linear-weighted composite · research preview”. Removed every “AI” / “ML” / “machine learning” wording in frontend/ and README.md. Tier-B and calibrated endpoints still return full data for internal users.
7. Visual polish	5 designer items: (a) accent colours desaturated to HSL S=60-75%, (b) first-screen only shows 4 indicator panels (SMA, RSI, MACD, Bollinger); the other 8 collapse into .more-indicators <details>, (c) brand stack: “IOSG” 18 px bold + “Tech Dashboard” 11 px upper-case, (d) Reversal badge no longer purple (neutral text-primary), (e) font sizes collapsed to 5 tiers (10 / 12 / 14 / 38 / 56).
8. PM patches	Favicon + social meta, case-insensitive ?token=ETH URL handling with toast, graceful 404 / token-not-found state.

Explicit out-of-scope for Phase 3 (do not work on these without authorization):

- Changing the Tier-A 0.40 trend weight.
- Exposing that the 5d Spearman correlation is statistically reversed (user decision: keep silent in UI).
- Adding survivorship-bias UI banner.
- Point-in-time universe rebuild (Phase 3 backlog).
- Rate-limit / proper user-auth system (the IOSG access code is sufficient for the controlled-preview model).
- Backend token validation (front-end gate is sufficient).

3. Functional modules — feature inventory

3.1 Frontend modules (crypto-tech-dashboard/frontend/)

Module	File	Purpose
Access gate	login.html, js/auth.js	Password page; localStorage token.
Main SPA	index.html (52 KB), js/app.js (~73 KB / 1386 LOC)	Topbar, sidebar tabs (Crypto / US Stocks), token search combobox, all chart wiring, theme toggle, score panels, robustness section, market overview, footer.
	js/api.js	Fetch helpers with relative URL base.

Module	File	Purpose
API wrapper		
Candle chart	js/charts/candle.js	Primary OHLCV chart with volume overlay; drives time-axis sync.
Indicator panels	js/charts/indicator_panels.js	12 panels, each with its own Lightweight Charts instance, slaved to candle's subscribeVisibleLogicalRangeChange.
Score gauge	js/components/score_gauge.js	SVG arc + needle for Trend / Reversal / Overall.
Sparkline	js/components/sparkline.js	Sidebar rank-row mini chart.
Market panel	js/components/market_panel.js	Renders mcap rank, mcap value, 24h volume, 30d avg volume.
Robustness panel	js/components/robustness_panel.js	Strategy reliability table with badge colours.
Explainer modal	js/components/explainer_modal.js	Modal popover for “What is Trend Score?” pulled from /api/scoring/explainer.
Toast	js/components/toast.js	Transient UI messages.
Vendored library	lib/lightweight-charts.standalone.production.js	TradingView Lightweight Charts v4.2.0 (pinned).
Styles	css/styles.css, css/login.css	Dark + light themes via html[data-theme="..."] attribute.

3.2 Backend modules (crypto-tech-dashboard/backend/)

Layer	Files
Entrypoint	main.py (479 LOC)
Config	config.py
Data — clients	data/exchange_client.py, data/coingecko_client.py, data/yfinance_client.py
Data — mapping	data/symbol_mapping.py
Data — store	data/local_store.py
Data — validator	data/data_validator.py

Layer	Files
Data — integrity	data/integrity.py
Data — exclusion	data/exclusion.py
Data — fetcher	data/fetcher.py (1433 LOC)
Indicators	indicators/ {base,registry,ma_cross_sma,ma_cross_ema,macd,rsi,rsi_mr,kdj,bollinger,
Scoring	scoring/{trend_score,reversal_score,overall_score,calibrated_weights,ex
Backtest	backtest/{engine,strategies,golden_cross,universe_robustness}.py
Services	services/data_service.py (782 LOC)
API	api/routes_{tokens,indicators,scores,backtest,system,admin,market,robust

3.3 Scripts (crypto-tech-dashboard/scripts/)

Script	Purpose
setup.sh	One-shot: create venv/, install requirements.txt, copy .env.example → .env, make logs/.
full_initial_load.py	Cold-start: pull Top-200 + ~3-year OHLCV (Phase 1 default; HISTORY_DAYS now 2326 for ~6y back to 2020-01-01).
initial_small_fetch.py	5-token smoke-test cold start.
run_history_extension.py	Append-extend mode: walk each token's earliest date backward until

Script	Purpose
	2020-01-01 or listing date.
<code>run_stocks_history_extension.py</code>	Same idea for the 40 stocks via yfinance.
<code>backfill_scores_history.py</code>	Reconstruct <code>local_data/market_cap/scores_history.csv</code> from existing OHLCV (~5-15 min).
<code>backfill_mcap_daily.py</code>	Reconstruct <code>market_cap_daily.csv</code> .
<code>backfill_overall_score.py</code>	Recompute the Overall composite over the historical scores frame.
<code>refresh_stocks_market.py</code>	Stand-alone refresh of <code>local_data/metadata/stocks_market.json</code> .
<code>analyze_horizons.py</code> (35 KB)	Walk-forward ρ analysis across 5d/10d/20d horizons; informs calibrated weights.
<code>train_tier_b.py</code> (17 KB)	Tier-B Ridge regression training + accept gate.
<code>pack_green_folder.sh</code>	Zip everything except <code>venv/</code> , <code>.git</code> , backups, the 47 MB pickle cache; redacts CoinGecko key from <code>.env</code> .
<code>install_launchd.sh</code> + <code>.plist.template</code>	macOS LaunchAgent installer for native (non-Docker) run.
<code>smoke_{data_layer,indicators,api,frontend}.py</code>	Per-layer smoke tests.
<code>integration_test.py</code>	End-to-end happy-path.

4. Data research workflow & data sources

This section is the new maintainer's mental model of how data flows. **Read once carefully** — it explains the trade-offs that drove the architecture.

4.1 Universe construction

CoinGecko Pro `/coins/markets` (3 pages $\times$ 250 = 750 candidates, sorted by `market_cap`)



```

exclusion.is_excluded(coin)
    | keyword filter → drop any name/symbol containing
    | "dai", "tusd", "usdd", "weth",
    | "usdp", "gusd", "lusd", "fdusd",
    | "sUSD", "eUSD", "wrapped", "wbtc",
    | "renbtc", "staked", "stake"
    | id blacklist → ~25 hand-picked ids (bridged-
wrapped-*, sbtc-2,
    | binance-peg-*, ousg, tether-gold,
    | kinesis-*,
    | spiko-*, vaneck-treasury-fund,
    | etc.)
    ▼
take first 200 survivors → local_data/market_cap/
top200_current.csv
                                (cg_id, symbol, name, price, mcap,
mcap_rank, ...)

```

The stocks universe is **hand-curated**: 40 US tickers in `local_data/metadata/stocks_universe.csv`, with `asset_class=us-stock`. No automatic discovery. To add or remove a stock, edit the CSV and restart.

4.2 Symbol mapping

For each crypto `cg_id`, `SymbolMapper.resolve(cg_id)` walks the 8-exchange priority list and asks CCXT `load_markets()` for `{SYMBOL}/USDT`. The first hit wins. Manual overrides go into `local_data/metadata/symbol_map.json`:

```

{
  "polygon-ecosystem-token": {"exchange": "binance", "symbol":
    "POL/USDT", "method": "manual"},
  "hashnote-usyc": {"exchange": "coingecko", "symbol":
    null, "method": "fallback", "reason":
    "not_on_any_exchange"}
}

```

4.3 OHLCV fetch — the 8-exchange waterfall

Exchange priority is hard-coded in `config.py:91-94`:

```

EXCHANGE_PRIORITY = [
    "binance", "okx", "bybit", "gateio",
    "coinbase", "kraken", "kucoin", "bitstamp",
]

```

Per-exchange notes:

Exchange	Public REST	Auth	Per-call cap	Notes
Binance		none	1000 bars	

Exchange	Public REST	Auth	Per-call cap	Notes
	/api/v3/klines			HTTP 451 from mainland China / Hong Kong — silently dropped from waterfall.
OKX	/api/v5/market/candles	none	100 bars	Workhorse from blocked regions.
Bybit	/v5/market/kline	none	1000 bars	HTTP 403 from CN/HK — dropped.
Gate.io	/api/v4/spot/candlesticks	none	1000 bars	
Coinbase	(CCXT default)	none	300 bars	Added Phase-2.
Kraken	(CCXT default)	none	720 bars	Added Phase-2.
KuCoin	(CCXT default)	none	1500 bars	Added Phase-2.
Bitstamp	(CCXT default)	none	1000 bars	Added Phase-2.

The fetcher walks the `since=` cursor forward in pages until it reaches “now”, then dedupes by date and clips to the most recent `HISTORY_DAYS` bars. **If you change `HISTORY_DAYS`, also re-run `scripts/run_history_extension.py` to backfill the new days for every token.**

At boot, `lifespan` calls `load_markets_all()` and writes per-exchange `available: bool + markets_count` into `last_update.json` → `exchange_health`. The `/api/system/status` endpoint surfaces this so the UI can render a “degraded” badge when an exchange is geo-blocked.

4.4 CoinGecko close-only fallback (Tier 4)

When all 8 exchanges fail (about 5-15 % of universe depending on the day):

- `coingecko_client.fetch_close_price_history(cg_id, days)` calls `coins/{cg_id}/market_chart/range`.
- Fills `open = high = low = close, volume = 0, source = "coingecko"`.
- KDJ + Volume Spike auto-NaN-guard for these rows.
- UI shows a “close-only data” badge for these tokens.

CoinGecko T+1 offset detection: on every boot, `coingecko_client.validate_cg_offset()` compares 30 days of BTC closes from CG vs OKX; if the implied offset is non-zero, it persists `local_data/metadata/cg_offset.json` and the close-only fetch path automatically date-shifts.

4.5 Data coverage metadata

Per-token tiers and gaps are tracked in `local_data/metadata/data_coverage.json`:

```
{
  "bitcoin": {
    "earliest_date": "2020-01-02",
    "latest_date": "2026-05-18",
    "listing_date": "2009-01-09",
    "real_ohlc_from": "2020-01-02",
    "close_only_windows": [],
    "tier_breakdown": [
      {"from": "2020-01-02", "to": "2026-05-18", "tier": 1,
       "source": "binance", "rows": 2329}
    ]
  }
}
```

This drives the **Data Coverage** collapsible chip near the Score area: “Exchange OHLC from 2020-01-02 · KDJ/Volume valid from 2020-01-02 · Tier-1 binance throughout”.

4.6 Incremental write & corruption recovery

- `LocalStore.append_ohlc()` does `pd.concat([existing, new]).drop_duplicates(subset=['date'], keep='last')` → write to `.tmp` → `os.rename()`. **Atomic** on the same filesystem.
- `data/integrity.py` runs at lifespan boot. For every CSV under `local_data/ohlc/`:
 1. Non-zero file size.
 2. Header equals canonical OHLCV columns.
 3. `pd.read_csv` succeeds.
 4. Row count $\geq$ `MIN_OHLCV_ROWS` (30).
 5. Last date within 14 days for active tokens.
 6. `validate_ohlc` issues empty.
- Failing files are **quarantined** (moved to `local_data/quarantine/`, not deleted).
- `POST /api/admin/repair/{cg_id}` (localhost-only — `request.client.host == "127.0.0.1"`) re-pulls the token via the full waterfall and atomically writes a fresh CSV.
- Full-load runs first snapshot `ohlc/` → `ohlc_backup_YYYYMMDD/` and keep the most recent `BACKUP_KEEP=3`.

4.7 Daily refresh flow

APScheduler 09:00 Asia/Shanghai

|



```
fetcher.run_daily_update()
    1. status = "updating"
    2. CoinGecko /coins/markets → new top200_current.csv (capture
new entrants)
    3. for each cg_id in store:
        ohlcv_csv = read existing
        days_needed = today - last_date + 2-day buffer
        new_rows = exchange_client.fetch_ohlcv_waterfall(symbol,
days=days_needed)
        OR
coingecko_client.fetch_close_price_history(cg_id, ...) (fallback)
        store.append_ohlcv(cg_id, new_rows)
    4. mcap snapshot to market_cap_daily/YYYY-MM-DD.csv
    5. data_validator over all appended files →
data_integrity_log.json
    6. snapshot trend/reversal scores → scores_history.csv (append
1 day × N tokens)
    7. status = "idle"
```

09:05 → `fetcher.run_stocks_daily_update()` runs the equivalent for the 40 stocks via yfinance.

4.8 Scores-history persistence

`local_data/market_cap/scores_history.csv` has columns `date`, `coin_id`, `trend_score`, `reversal_score`. One row per token per day. The 2y / 3y time-series percentiles read from this file. If it gets corrupted or rebased, `scripts/backfill_scores_history.py` reconstructs it from the OHLCV cache (~5-15 min).

5. Scoring methodology & experiments

This is the IP layer. The new maintainer should treat formulas and weights as **load-bearing** and not change them without a written audit trail.

5.1 12 indicator families

All formulas are ported verbatim from the source notebook (`compute_features` method in the final cell of `1_First_or_not_run___Tech_Fac_Price_Mcap_volumeData.ipynb`), except KDJ which is new. Default parameters:

#	Family	Default params	Key output keys
1	SMA Cross	fast=5, slow=20	

#	Family	Default params	Key output keys
			sma_cross_strength_signed_5_20, sma_cross_up/down
2	EMA Cross	fast=5, slow=20	ema_cross_strength_signed_5_20, ema_cross_up/down
3	MACD	(12, 26, 9)	macd_hist_12_26_9, macd_hist_slope5_12_26_9, macd_cross_event
4	RSI	period=14, Wilder smoothing ewm(alpha=1/period, adjust=False)	rsi_14, rsi_dist_os_14, rsi_turn_event_14
5	RSI Mean-Reversion	period=14	rsi_mr_dist_os_14, rsi_mr_dist_os_14_clip (note: rsi_dist_os_14 without _mr_ prefix lives in family #4)
6	KDJ	N=9, M1=3, M2=3 (initialised K0=D0=50)	kdj_k/d/j, kdj_os_distance, kdj_golden_cross
7	Bollinger	period=20, std=2	bb_pctb_20, bb_z_20, bb_width_20
8	Volume Spike	ma_window=14	vol_ratio_14, vol_z_14, vol_spike_3x_14
9	Momentum	windows=[5, 10, 20, 30]	mom_ret_5d/10d/20d/30d
10	Mean Reversion (skip)	L=40, S=16	mr_z_40_skip16, mr_rank_40_skip16
11	Z-Score vs MA50	ma=50, z_windows=[20, 40, 80, 120] (scoring layer uses _40)	ma50_dev, ma50_dev_z_40, ma50_slope_20d, ma50_cross_up/ dn
12	Price Appreciation	threshold=5 %, ret_windows=[3,5,10,20], vol_ma_windows=[7,14,21]	price_ret_20d, price_app_5pct_10d, vol3x_and_price5_14_10d

Critical gotchas:

- RSI **must** use Wilder's ewm(alpha=1/period, adjust=False). SMA-based RSI will diverge from TradingView by several points. Test: BTC RSI(14) on a known date matches TradingView within 0.5.
- MACD line is **normalized to price**: (ema_fast - ema_slow) / Close. Don't strip the divisor.
- CCXT symbol format is BTC/USDT (with slash). The unslashed BTCUSDT will not match anything.
- CSVs are named by CoinGecko ID (e.g. bitcoin.csv, ondo-finance.csv), **not** exchange symbol.
- Atomic write is mandatory for append_ohlcv. Never write the CSV directly.

5.2 Trend score (9 signals)

Code: [backend/scoring/trend_score.py:24-34](#).

For each signal, take the cross-sectional rank-percentile across today's universe (0-100), then equal-weight average:

```
Trend = mean(percentile_rank_cs([
    mom_ret_10d, mom_ret_20d,
    macd_hist_12_26_9, macd_hist_slope5_12_26_9,
    sma_cross_strength_signed_5_20, ema_cross_strength_signed_5_20,
    ma50_slope_20d, ma50_dev, bb_pctb_20
]))
```

5.3 Reversal score (7 signals)

Code: [backend/scoring/reversal_score.py:21-29](#).

Same cross-sectional percentile + equal-weight blend, with -1 sign applied to bb_z_20, ma50_dev_z_40, and mom_ret_5d (high values of those = momentum, not reversal):

```
Reversal = mean(percentile_rank_cs([
    rsi_dist_os_14, rsi_turn_event_14,
    kdj_os_distance, -bb_z_20,
    mr_z_40_skip16, -ma50_dev_z_40, -mom_ret_5d
]))
```

5.4 Overall composite (Tier A, finance-theory priors)

Code: [backend/scoring/overall_score.py:44-51](#).

```
Overall = 0.40 · Trend                (cross-sectional
percentile)
        + 0.25 · Reversal            (cross-sectional
percentile)
        + 0.15 · Breadth              (CS-rank of % of 9 trend
signals > 0)
        + 0.10 · Risk                 (CS-rank of inverse 20d
vol)
        + 0.05 · TS_Trend_2y          (this token's 2-y rolling
self-percentile)
        + 0.05 · TS_Reversal_2y       (same for reversal_score)
```

Weight rationale (anchored to Liu/Tsyvinski 2021 “Risks and Returns of Cryptocurrency” + Russell/Engle 2010):

- 0.40 Trend: momentum is the most empirically robust factor in crypto.
- 0.25 Reversal: real but noisier; bounces back into a downtrend a lot.
- 0.15 Breadth: confirmation discount — multi-signal agreement.
- 0.10 Risk: penalises high-vol moonshots, Sharpe-like.
- 0.05 + 0.05 TS sleeves: capture rare-strength outliers that recent CS-only scores miss.

Notational footnote: the Phase-2 plan writes the TS sleeves as $0.10 \times \text{TS_Trend_2y} \times 0.5 + 0.10 \times \text{TS_Reversal_2y} \times 0.5$. That is mathematically the same as $0.05 + 0.05$ — the production code in `overall_score.py:44–51` persists them as 0.05 each.

Tier-A walk-forward accept gate: Tier-A itself was validated against a baseline of $(\text{Trend} + \text{Reversal}) / 2$ with the requirement $\rho(\text{Overall}, \text{fwd_5d_return}) \geq \rho((\text{Trend} + \text{Reversal}) / 2, \text{fwd_5d_return}) + 0.05$. The gate passed during Phase 2; if the data foundation changes materially (e.g. a multi-year universe rebuild), re-run the walk-forward analysis in `scripts/analyze_horizons.py` before committing new weights.

5.5 Tier B (Ridge regression, experiment)

`scripts/train_tier_b.py` runs a pooled-panel Ridge regression with date-fixed effects:

- 24 months train / 1 month test / monthly rolling walk-forward.
- 16 atomic signal CS-percentiles + 4 sleeve percentiles as features.
- Target: 5-day forward log return per token.
- `RidgeCV(alphas=[0.1, 1, 10, 100])`.
- 12-fold sign-stability check: drop any coefficient that flips sign.

Accept gate: holdout Spearman $\rho \geq$ Tier-A baseline + 0.02. If accepted, weights write to `local_data/scoring/tier_b_weights.json` with `accept: true` and `/api/scoring/tier_b` returns them. The frontend toggle shows a Tier-B option. **As of last training, the gate did not pass;** the toggle stays hidden and Tier-A is the production path.

5.6 Calibrated weights (third option)

`backend/scoring/calibrated_weights.py` sizes each sleeve weight by $|\text{Fama-MacBeth } \rho|$ against the forward 5d return and respects empirical sign. Sleeves whose sign disagrees with their intent can be either **dropped** or **flipped**, controlled by a config knob. Stored in `local_data/scoring/calibrated_weights.json`. Same fall-back behaviour: missing/invalid $\rightarrow$ Tier-A.

5.7 Indicator robustness backtests

For each of 9 canonical strategies (RSI 30-50, MACD signal cross, KDJ oversold, Bollinger lower-band, SMA/EMA golden cross, momentum breakout, z-score reversion, price appreciation), the engine runs each strategy on every token in the universe. Aggregations:

- median Sharpe
- mean Sharpe
- % of tokens with positive Sharpe
- worst case (Sharpe, `cg_id`)
- best case (Sharpe, `cg_id`)

Reliability badge (calibrated against BTC buy-hold Sharpe ≈ 1.1):

Badge	Median Sharpe	% Positive	Worst
reliable	≥ 0.5	$\geq 60\%$	≥ -1.0
caveats	≥ 0.2 OR $\geq 50\%$		
unreliable	< 0.2 AND $< 50\%$		

Results cached at `local_data/robustness_cache/<sha256>.json`; cache key is sha256 of sorted (cg_id, size, mtime) across `local_data/ohlcv/`.

Recompute: `POST /api/indicator-robustness/recompute`.

5.8 Score-explainer JSON

`backend/scoring/explainers.py` builds the `/api/scoring/explainer` payload — title, one-line summary, full formula in markdown, current signal table (live values), strengths, weaknesses, and band interpretations (`above_70 / 33_70 / below_33`). The frontend modal renders this via `js/components/explainer_modal.js`.

6. Complete API surface

All endpoints are mounted at the FastAPI root. The frontend uses relative URLs.

Method	Path	Purpose
GET	/	SPA index (frontend/index.html).
GET	/login.html	Access-code gate.
GET	/health	<code>{"ok": true}</code> — used by <code>start.command</code> healthcheck.
GET	/favicon.ico	Real favicon or 1×1 transparent PNG.
GET	/api/tokens	All tracked tokens with metadata. Supports <code>?asset_class=crypto us-stock</code> .
GET	/api/tokens/{cg_id}	One token + last close + <code>close_only_data</code> flag.
GET	/api/token/{cg_id}	Singular alias of above (Phase-1 audit fix).
GET	/api/ohlc/{cg_id}?days=365	OHLCV time series.
GET	/api/sparklines?ids=a,b,c&days=30	Batch close-only series for sidebar mini-charts.
GET	/api/indicators/{cg_id}?days=365	All 12 families' time series for one token.
GET		

Method	Path	Purpose
	/api/indicators/{cg_id}/{family}?days=365&fast=5&slow=20	One family with parameter overrides.
GET	/api/scores?asset_class=crypto&limit=200	All tokens' trend / reversal / overall scores.
GET	/api/scores/{cg_id}	One token's full score block + CS rank + 2y/3y TS percentile.
GET	/api/rankings?sort_by=trend\ reversal\ overall&limit=20&asset_class=crypto	Sorted sidebar feed.
GET	/api/backtest/{cg_id}?fast=5&slow=20	Golden-cross backtest stats + equity curve.
GET	/api/market_overview/{cg_id}	Mcap rank, mcap, 24h volume, 30d avg volume, liquidity proxy.
GET	/api/indicator-robustness?asset_class=crypto	All 9 strategies' aggregated badges.
GET	/api/indicator-robustness/{strategy_name}	Per-strategy detail with per-token Sharpe.
POST	/api/indicator-robustness/recompute	Invalidate cache and rerun.
GET	/api/scoring/explainer	Full explainer dict for Trend / Reversal / Overall.
GET	/api/scoring/explainer/{kind}	One of trend / reversal / overall.
GET	/api/scoring/calibrated	Calibrated-weights JSON (research).
GET	/api/scoring/tier_b	Tier-B Ridge weights JSON (research).
GET	/api/system/status, /api/status	Last-update timestamp, token count, exchange health, status.
GET	/api/system/health	Server health + version.
POST	/api/system/refresh?full=false, /api/refresh	Trigger daily update (or full reload with full=true).
GET	/api/system/refresh-progress	Progress polling for long refresh runs.
GET	/api/data-check	Run validate_ohlcv + validate_top200 inline.
GET	/api/data-coverage, /api/data-coverage/{cg_id}	Per-token tier breakdown + coverage windows.
GET	/api/admin/integrity	Last boot integrity report. Localhost-only.

Method	Path	Purpose
POST	/api/admin/repair/{cg_id}	Re-pull and atomically rewrite one CSV. Localhost-only.

7. Local-mode quickstart (no Docker)

For development on a Mac without Docker. Production lives in the Docker container described in §8.

```
cd crypto-tech-dashboard-2nd-try-v2.0/crypto-tech-dashboard
```

```
# 1. One-time setup
./scripts/setup.sh           # creates venv/, pip install -r
                             requirements.txt,
                             # copies .env.example -> .env, makes
                             logs/

# 2. Configure secrets – replace the placeholder CoinGecko Pro key
$EDITOR .env                 # set COINGECKO_API_KEY=cg_live_...

# 3. Run
./run.sh                     # binds 127.0.0.1:8080 (default)
./run.sh --public             # binds 0.0.0.0:8080 (LAN-
                             accessible)
./run.sh --port 9000          # custom port
```

Then open <http://localhost:8080/>. First boot with empty `local_data/` shows nothing useful — trigger the cold load:

```
curl -X POST 'http://localhost:8080/api/system/refresh?full=true'
# expect ~7 minutes; logs go to the foreground uvicorn console.
```

After the first load, subsequent boots reuse the existing CSVs and are near-instant. The 09:00 cron will append one new row per token each morning.

7.1 macOS LaunchAgent (optional autostart)

```
bash scripts/install_launchd.sh
# logs at logs/uvicorn.{out,err}.log
# uninstall:
launchctl unload -w ~/Library/LaunchAgents/com.iosg.crypto-
dashboard.plist
rm ~/Library/LaunchAgents/com.iosg.crypto-dashboard.plist
```

8. Docker deployment (production)

This is the path used in production. The Mac mini that hosts the public preview runs the container plus a Cloudflare Named Tunnel.

8.1 Files involved

File	Role
Dockerfile	Image build — python:3.12-slim, install requirements.txt, copy backend/frontend/local_data, run uvicorn backend.main:app.
docker-compose.yml	One service dashboard, mounts ./local_data as a bind volume, sets TZ=Asia/Shanghai, restart: unless-stopped.
.env	Travels inside the folder with the zip (it's read at runtime by env_file:, not baked into the image — .dockerignore excludes it from the build context). Must contain a real COINGECKO_API_KEY.
.dockerignore	Excludes .env, venv/, .git, logs, local_data/ohlcv_backup_*/, local_data/quarantine/, the 47 MB indicator pickle cache.
start.command	Double-clickable launcher for non-engineers. Preflights Docker daemon, builds & starts container, waits up to 90 s for /health.
stop.command	Double-clickable shutdown. Runs docker compose down. The named tunnel keeps running independently.

8.2 Image build context

The build context is the crypto-tech-dashboard/ directory. The Dockerfile:

```
FROM python:3.12-slim
ENV PYTHONDONTWRITEBYTECODE=1 \
    PYTHONUNBUFFERED=1 \
    PIP_NO_CACHE_DIR=1 \
    DATA_DIR=/app/local_data \
    HOST=0.0.0.0 \
    PORT=8000
WORKDIR /app
RUN apt-get update \
    && apt-get install -y --no-install-recommends ca-certificates \
    curl tzdata \
    && rm -rf /var/lib/apt/lists/*
```

```

COPY requirements.txt .
RUN python -m pip install --upgrade pip setuptools wheel \
    && pip install -r requirements.txt
COPY backend ./backend
COPY frontend ./frontend
COPY local_data ./local_data
COPY README.md run.sh .env.example ./
EXPOSE 8000
CMD ["sh", "-c", "python -m uvicorn backend.main:app --host $
    {HOST:-0.0.0.0} --port ${PORT:-8000}"]

```

The local_data/ seed inside the image is a starting point; the runtime bind mount overrides it with the host's ./local_data, which is the **source of truth**.

8.3 docker-compose.yml

```

services:
  dashboard:
    build: .
    image: iosg-crypto-tech-dashboard:local
    container_name: iosg-crypto-tech-dashboard
    env_file:
      - .env
    environment:
      DATA_DIR: /app/local_data
      HOST: 0.0.0.0
      PORT: 8000
      TZ: Asia/Shanghai # so APScheduler 09:00 cron
                        matches operator wall-clock
    ports:
      - "8000:8000"
    volumes:
      - ./local_data:/app/local_data # daily-cron writes
                                      persist on host
    restart: unless-stopped

```

8.4 First-time setup on the destination Mac

```

# 1. Install Docker Desktop
brew install --cask docker
# (or download from docker.com)

# 2. Unzip the dashboard folder
unzip dashboard_green_YYYYMMDD.zip -d crypto-tech-dashboard
cd crypto-tech-dashboard

# 3. Verify .env is present and edit the key if needed
cat .env
    # COINGECKO_API_KEY=... must be a real Pro key
$EDITOR .env # rotate if shared zip used a
              placeholder

```

```
# 4. Build + start
docker compose up -d --build
# first build pulls python:3.12-slim and installs requirements:
# ~3-5 min
# subsequent starts: < 5 sec.

# 5. Open in browser
open http://localhost:8000
# (access code: IOSG – uppercase, 4 letters)
```

8.5 Day-to-day operator commands

```
docker compose up -d          # start (uses last build)
docker compose logs -f dashboard # tail logs
docker compose restart        # pick up code change after a
    rebuild
docker compose down           # stop + remove container
    (data preserved on host)
docker compose up -d --build   # rebuild after code change
```

8.6 The double-click path (non-engineer friendly)

start.command is a wrapper that:

1. Verifies Docker CLI is installed; opens Docker Desktop if the daemon isn't running; waits up to 2 min for the daemon.
2. Runs `docker compose up -d --build`.
3. Polls `http://localhost:8000/health` up to 90×2 s for a 200 response.
4. Prints the local URL + the access code; keeps the Terminal window open until Enter.

stop.command runs `docker compose down`. Both files have the executable bit set; if Spotlight indexing strips it, run `chmod +x start.command stop.command`.

8.7 Public access (Cloudflare Named Tunnel)

Production exposes the container at a permanent URL via a Cloudflare Named Tunnel managed by `launchd`. The full procedure is documented in [crypto-tech-dashboard/NAMED_TUNNEL_SETUP.md](#); the short version:

```
brew install cloudflared
cloudflared tunnel login          # opens
    browser, picks a CF zone
cloudflared tunnel create iosg-tech-dashboard # writes
    ~/.cloudflared/<uuid>.json
cloudflared tunnel route dns iosg-tech-dashboard tech.iosg.vc #
    binds custom hostname
# Write ~/.cloudflared/config.yml:
#   tunnel: iosg-tech-dashboard
#   credentials-file: /Users/<user>/.cloudflared/<uuid>.json
#   ingress:
#     - hostname: tech.iosg.vc
```

```
#      service: http://localhost:8000
#      - service: http_status:404
cloudflared tunnel run iosg-tech-dashboard           # test
      once
sudo cloudflared service install                      #
      register as launchd, auto-restart
```

Verification:

```
ps aux | grep cloudflared
sudo launchctl list | grep cloudflared
tail -f /Library/Logs/com.cloudflare.cloudflared.out.log
```

The tunnel is **independent of the container**. If the container goes down the public URL returns 502; if cloudflared crashes, launchd respawns it; if the Mac reboots, both come back automatically.

8.8 Volumes & data persistence

The bind mount `./local_data:/app/local_data` means:

- The host folder is the source of truth.
- `docker compose down` does **not** delete data.
- To wipe and re-cold-load: `rm -rf local_data/ohlc/* && docker compose restart && curl -X POST http://localhost:8000/api/system/refresh?full=true`.

8.9 Sharing the green-folder zip safely

```
bash scripts/pack_green_folder.sh
# writes dashboard_green_YYYYMMDD.zip in the project root.
# Excludes: venv/, .git/, __pycache__, ohlc_backup*/, indicator
# pickle cache.
# Redacts COINGECKO_API_KEY in .env to a placeholder.
```

Safety rules:

-  Shipping the zip to a trusted teammate's Mac is fine — `.env` is only read at runtime, never baked into the image.
-  Never publish the resulting Docker image publicly.
-  Never push the zip to a public GitHub repo or S3 bucket — the CoinGecko Pro key inside `.env` would leak (unless you re-redacted just before sharing).

9. Operations & monitoring

9.1 Health endpoints

Probe	Endpoint
Container alive	GET /health

Probe	Endpoint
Last update + freshness	GET /api/system/status
Exchange waterfall health	GET /api/system/status → exchange_health. {exchange}.available
Per-token integrity	GET /api/admin/integrity (localhost-only)
Per-token data coverage	GET /api/data-coverage/{cg_id}

9.2 Smoke test (run first when something looks wrong)

```
curl -s localhost:8000/health # {"ok": true}
curl -s localhost:8000/api/system/status | jq
curl -s 'localhost:8000/api/scores?limit=5' | jq
curl -s 'localhost:8000/api/indicators/bitcoin/rsi?period=14' | jq '.params'
curl -s 'localhost:8000/api/ohlc/MSTR?days=30' | jq '[-1]' # US stocks
```

Frontend visual check (open <http://localhost:8000>):

- Access gate accepts I0SG.
- Token combobox suggests on type (search by symbol / name / id).
- Candle chart fills the top panel; first-screen shows 4 indicator panels (SMA, RSI, MACD, Bollinger); “Show more indicators” expands the rest.
- Overall hero card on top with composite gauge; Trend + Reversal cards below.
- Sidebar tabs [Crypto (200)] [US Stocks (40)]; Top-20 has rank, symbol, sparkline, score.
- Footer: copyright, version, last-update relative time, disclaimer.

9.3 When the daily refresh fails

Symptoms: /api/system/status shows last_ohlc_update ≥ 24 h old; topbar reads “Updated 1 day ago” or more.

Diagnostic order:

1. docker compose logs --tail=200 dashboard — look for auto-refresh crypto failed: / self-heal stocks failed:.
2. Check /api/system/status → exchange_health — is any exchange unavailable?
3. Verify CoinGecko key isn’t revoked: curl -s -H "x-cg-pro-api-key: \$KEY" https://pro-api.coingecko.com/api/v3/coins/markets?vs_currency=usd&per_page=1.
4. If CoinGecko 401/403 → rotate .env key, docker compose restart.
5. If a single token’s CSV is wedged: curl -X POST 'http://localhost:8000/api/admin/repair/<cg_id>' (from inside the host).

9.4 When a CSV is corrupt

- It's already quarantined to `local_data/quarantine/`.
- `local_data/metadata/data_integrity_log.json` has the details.
- Run `POST /api/admin/repair/{cg_id}` to re-pull. The quarantine file is left in place as evidence.

9.5 When the host machine slept through 09:00

The `misfire_grace_time = 6 h + coalesce = True` configuration means APScheduler catches up exactly once on wake — no manual intervention needed for short overnight sleeps. If the gap is > 6 h, the hourly self-heal cron at `:30` will pick it up within an hour (4 h cooldown).

9.6 Rotating the CoinGecko API key

```
$EDITOR .env # replace
COINGECKO_API_KEY
docker compose restart # `.env` is read at
start, not build
# the boot-time CG offset detection runs against the new key
```

9.7 Adding / removing a stock ticker

```
$EDITOR local_data/metadata/stocks_universe.csv
# add or comment out a row
docker compose restart
curl -X POST 'http://localhost:8000/api/system/refresh?full=false'
```

9.8 Adding / removing a crypto exclude

```
$EDITOR local_data/metadata/crypto_exclude.txt # one cg_id per
line, '#' = comment
docker compose restart
```

10. Known limits & open items

These are **deliberately deferred**, not bugs. The new maintainer should be aware before promising features to stakeholders.

Area	Status	Notes
Tier-B Ridge holdout	Did not pass the +0.02 accept gate as of last training. UI hides the Tier-B toggle. Re-run <code>python scripts/train_tier_b.py</code> after enough new history accumulates.	

Area	Status	Notes
Tier-A 0.40 trend weight	User decision: keep as-is even though 5d Spearman is ~zero. Audit copy avoids exposing this.	
Survivorship bias	The universe is <i>current</i> Top-200 by mcap. Tokens that delisted are gone from history. Phase-3 backlog item: point-in-time universe rebuild.	
Hong Kong stocks	Skipped. Phase-3 backlog.	
Backend auth	None. The front-end IOSG password gate is the only access control — sufficient for the controlled-preview model, but do not expose the Docker port directly to the internet without an upstream proxy.	
Rate limiting	None on the backend. Behind the Cloudflare Named Tunnel; rely on Cloudflare for ddos basics.	
pytest suite	Not present. Smoke tests in <code>scripts/smoke_*.py</code> cover the basic paths. Add proper pytest CI when the team has bandwidth.	
Mobile drawer	Sidebar collapses below 900 px but is not a slide-up drawer. Cosmetic.	
Score-component label humanisation	Done in Phase 2 for the headline labels; raw keys still appear in some breakdown rows. Cosmetic.	
Indicator pickle cache	47 MB at <code>local_data/scoring/_horizon_panel_cache.pkl</code> . Excluded from the green-folder zip. Recomputed on demand by <code>analyze_horizons.py</code> .	

11. Repo layout reference

```

16.singal_coin_technical/                                     ← parent
folder (one level up)
├── _chinese_plans_archive/                                   ←
original Chinese plans (historical reference, kept outside the

```

```

deliverable)
├── PLAN_技[?]指[?]Dashboard.md
├── 二期Plan-技[?]指[?]Dashboard.md
├── 三期Plan-修改清[?].md
└── crypto-tech-dashboard-2nd-try-v2.0/
root (this folder)
├── HANDOVER.md
DOCUMENT
├── HANDOVER.pdf
rendering of THIS DOCUMENT
├── PLAN_Phase1_EN.md
plan (English translation)
├── PLAN_Phase2_EN.md
plan (English translation)
├── PLAN_Phase3_EN.md
plan (English translation)
├── PROGRESS.md
chronological build log (English)
├── PRODUCT_REVIEW.md
PM audit from end of Phase 1 (English)
├── PROJECT_FILES_INDEX.md
of root-level files (English)
├── README_TASK.md
note from initial scaffold (English)
├── BUILD_STATUS.json
snapshot of build-phase metadata
├── 任[?]交接指南.md
Chinese hand-off guide for the *first* hand-off (superseded by
this white paper)
├── 1_First_or_not_run____..._volumeData.ipynb
notebook (formula reference)
├── run_praisonai.py + .praison-venv/
orphaned PraisonAI scaffold attempt; safe to delete
├── references/
hand-off material (one Chinese file)
├── output/
reserved for deliverables
├── screenshots/
screenshots from audits
├── crypto-tech-dashboard/
deployable application
├── README.md
ops README (English)
├── NAMED_TUNNEL_SETUP.md
Cloudflare named-tunnel doc (originally Chinese – consider
translating)
├── Dockerfile
├── docker-compose.yml
├── start.command
clickable launcher
├── stop.command
├── run.sh
launcher (no Docker)
├── requirements.txt

```

← project
 ← THIS
 ← PDF
 ← Phase-1
 ← Phase-2
 ← Phase-3
 ←
 ← Google-
 ← index
 ← short
 ←
 ← legacy
 ← source
 ←
 ← backup
 ←
 ← UI
 ←
 ← short
 ←
 ← double-
 ← uvicorn

— .env.example	← copy
→ .env, set COINGECKO_API_KEY	
— .dockerignore	
— docs/scoring_audit/	←
scoring-audit artefacts from Phase-2 audits	
— backend/	
— main.py	← FastAPI
app + APScheduler	
— config.py	← .env
loader + paths + exchange priority	
— api/	← 9 route
files + `_validators.py` regex allowlist (see §6)	
— data/	← clients
(CCXT, CG, yfinance), store, validator, fetcher, integrity	
— indicators/	← 12
families + base + registry + volatility	
— scoring/	← trend/
reversal/overall/calibrated/explainers/ranking	
— backtest/	← engine
+ strategies + universe_robustness + golden_cross	
— services/data_service.py	← in-
memory singleton cache	
— frontend/	
— index.html	← SPA
shell	
— login.html	← access-
code page	
— css/styles.css, css/login.css	
— js/{api,app,auth}.js	
— js/charts/{candle,indicator_panels}.js	
— js/components/	
{score_gauge,sparkline,market_panel,robustness_panel,explainer_mod	
— lib/lightweight-charts.standalone.production.js	
— local_data/	← runtime
cache (bind-mounted into container)	
— ohlcv/<cg_id>.csv	← one CSV
per token (240+ files)	
— market_cap/{top200_current,scores_history}.csv +	
mcap_daily/	
— metadata/	
{symbol_map,last_update,data_integrity_log,cg_offset,data_coverage,	
— metadata/stocks_universe.csv, crypto_exclude.txt	
— scoring/	
{tier_b_weights,calibrated_weights,horizon_calibration,holdout_wall	
— robustness_cache/<sha256>.json	←
strategy robustness results	
— ohlcv_backup/YYYYMMDD/	← rotated
snapshots (kept N=3)	
— quarantine/	←
unreadable CSVs moved here	
— scripts/	← see
§3.3 for the full table	

12. Glossary

Term	Meaning
cg_id	CoinGecko canonical token id (e.g. bitcoin, ondo-finance). Also used as the CSV filename and the primary key everywhere.
Tier-A weights	Finance-theory prior weights for the Overall composite (0.40 / 0.25 / 0.15 / 0.10 / 0.05 / 0.05).
Tier-B weights	Ridge-regression-trained weights from <code>scripts/train_tier_b.py</code> . Live only if the holdout accept-gate passes.
Calibrated weights	Sleeve weights sized by $ Fama-MacBeth\ \rho $, sign-respecting. Third option after Tier-A and Tier-B.
CS percentile	Cross-sectional percentile rank within today's universe (one snapshot).
TS percentile	Time-series rolling-window self-percentile (per-token, over its own 2y or 3y history).
Breadth	% of the 9 trend signals that are strictly positive, cross-sectionally ranked.
Risk sleeve	Inverse 20-day volatility, CS-ranked. Low vol $\rightarrow$ high score.
Robustness backtest	9 canonical strategies $\times$ every token in the universe; aggregated to a per-strategy reliable / caveats / unreliable badge.
Green folder	The portable zip produced by <code>scripts/pack_green_folder.sh</code> — a copy-paste deployable bundle.
Named tunnel	Cloudflare's permanent-URL tunnel mode, as opposed to the disposable Quick Tunnel. Documented in <code>NAMED_TUNNEL_SETUP.md</code> .

13. Contacts & escalation

Topic	Who
Original quant design + this hand-off	Zeun (leaving the team — reach out before the cut-off date for clarifications)
IOSG investment context	Jocy, Momir, Yiping (IC)
Portfolio support	Shawn

Topic	Who
IT / Mac mini / Cloudflare	Xiaonan
Operations	Roy

When something breaks in production, the **first** Slack to ping is the IT contact (Xiaonan) plus whichever IC member is using the dashboard at that moment. The Mac mini that hosts the container also runs the cloudflared launchd service — those two systems can fail independently.

End of hand-over document.